

AI-DRIVEN AUTOMATED WEB APPLICATION PENETRATION TESTING SYSTEM

COMPLETE TECHNICAL ARCHITECTURE & SYSTEM DESIGN SPECIFICATION

A Professional Engineering Blueprint for Security Researchers,
Red Team Engineers, and AI-Assisted Security Tooling Developers

Version	2.0 — Full Production Blueprint
Classification	Professional / Authorized Security Research
Domain	Cybersecurity / AppSec / AI Engineering
Audience	Security Engineers, Developers, AI Builders
Date	2025

⚠ FOR AUTHORIZED USE ONLY ⚠

This system must only be used against targets explicitly authorized for security testing.

Table of Contents

- Table of Contents 2
- 1. Project Overview..... 6
 - 1.1 The Cybersecurity Problem 6
 - 1.2 Limitations of Traditional Scanners..... 6
 - 1.3 Concept of Autonomous Penetration Testing 6
 - 1.4 How the System Simulates Attacker Workflows..... 6
 - 1.5 AI-Enhanced Vulnerability Discovery..... 7
 - 1.6 Real-World Applications 7
- 2. Research and Development Context 8
 - 2.1 State of Automated Security Testing 8
 - 2.2 Key Research Concepts..... 8
 - 2.2.1 Attack Surface Discovery 8
 - 2.2.2 Fuzz Testing 8
 - 2.2.3 Automated Reconnaissance 8
 - 2.2.4 Intelligent Payload Generation 8
 - 2.2.5 Response Anomaly Detection 8
 - 2.2.6 AI-Assisted Vulnerability Classification..... 8
 - 2.3 How AWAP-AI Advances the State of the Art 9
- 3. Core Platform Capabilities 10
 - 3.1 Reconnaissance & Discovery 10
 - 3.2 Crawling & Endpoint Discovery 10
 - 3.3 Attack & Vulnerability Testing..... 10
 - 3.4 Intelligence & Orchestration 11
 - 3.5 Platform & Operations 11
- 4. System Architecture..... 12
 - 4.1 Architecture Diagram (Text-Based) 12
 - 4.2 Module Specifications..... 12
 - 4.2.1 Target Input Manager 12
 - 4.2.2 Recon Engine 13
 - 4.2.3 Crawler Engine 13
 - 4.2.4 JavaScript Analyzer 13
 - 4.2.5 Parameter Discovery Engine..... 13
 - 4.2.6 Attack Engine..... 14
 - 4.2.7 AI Decision Engine..... 14
 - 4.2.8 Report Generator 14

- 5. AI Integration Strategy 15
 - 5.1 AI Component Map 15
 - 5.2 AI-Assisted Payload Generation..... 15
 - 5.3 Response Anomaly Detection 15
 - 5.4 Vulnerability Classification Pipeline 15
 - 5.5 Attack Path Reasoning 16
 - 5.6 Adaptive Scan Prioritization (Multi-Armed Bandit) 16
 - 5.7 Recommended Models and Libraries 16
- 6. Attack Module Framework 17
 - 6.1 Module Interface Contract 17
 - 6.2 Module Catalog 17
 - 6.3 Module Integration Flow 18
- 7. Payload Generation Engine 19
 - 7.1 Generation Strategies..... 19
 - 7.1.1 Static Wordlist Baseline 19
 - 7.1.2 Mutation Engine 19
 - 7.1.3 Context-Aware Generation..... 19
 - 7.1.4 LLM-Assisted Generation..... 19
 - 7.1.5 Adaptive Payload Refinement..... 20
 - 7.2 Payload Encoding Matrix..... 20
- 8. Response Analysis Engine 21
 - 8.1 Analysis Layers 21
 - 8.1.1 Error Pattern Recognition..... 21
 - 8.1.2 Response Structure Anomalies..... 21
 - 8.1.3 Reflection Detection..... 21
 - 8.1.4 Behavioral Analysis..... 21
 - 8.1.5 Timing Analysis..... 21
 - 8.1.6 Out-of-Band (OOB) Callback Analysis 22
- 9. Technology Stack 23
 - 9.1 Backend Services..... 23
 - 9.2 AI / ML Stack..... 23
 - 9.3 Frontend..... 23
 - 9.4 Data Storage 24
 - 9.5 Infrastructure 24
- 10. User Interface and User Experience 25
 - 10.1 Screen Inventory 25
 - Screen 1: Main Dashboard 25
 - Screen 2: Target Management 25
 - Screen 3: Live Scan Monitor 25

Screen 4: Vulnerability Finding Detail	25
Screen 5: Security Analytics	25
Screen 6: Report Builder.....	26
11. Security and Ethical Safeguards	27
11.1 Authorization Enforcement	27
11.2 Rate Limiting and Throttling.....	27
11.3 Data Handling.....	27
11.4 Scan Impact Minimization.....	27
12. Application Workflow.....	29
12.1 Workflow State Machine.....	29
12.2 Phase Descriptions	29
Phase 1: Target Input & Scope Verification (CREATED → SCOPE_VERIFIED)	29
Phase 2: Reconnaissance (SCOPE_VERIFIED → RECON)	29
Phase 3: Crawl & Mapping (RECON → CRAWL → MAPPING)	29
Phase 4: Attack Execution (MAPPING → ATTACK).....	29
Phase 5: AI Analysis (ATTACK → ANALYSIS).....	30
Phase 6: Reporting (ANALYSIS → REPORTING → COMPLETE)	30
13. Reporting System	31
13.1 Finding Schema	31
13.2 Report Templates.....	32
14. Development Roadmap	33
Week 1: Foundation (Days 1-7).....	33
Days 1-2: Project Setup.....	33
Days 3-4: Recon Engine.....	33
Days 5-7: Crawler Engine v1	33
Week 2: Attack Engine (Days 8-14).....	33
Days 8-9: Payload Engine & Parameter Discovery	33
Days 10-11: Core Attack Modules	33
Days 12-13: More Attack Modules	33
Day 14: Response Analysis Engine	34
Week 3: AI Layer (Days 15-21)	34
Days 15-16: ML Vulnerability Classifier.....	34
Days 17-18: Anomaly Detection.....	34
Days 19-20: LLM Integration.....	34
Day 21: Attack Graph Engine.....	34
Week 4: Platform & Polish (Days 22-30)	34
Days 22-23: Frontend Dashboard.....	34
Days 24-25: Reporting System	35
Days 26-27: Security Safeguards & Hardening.....	35

- Days 28-29: Integration Testing 35
- Day 30: Documentation & Release Prep..... 35
- 15. Future Extensions 36
 - 15.1 LLM-Based Autonomous Pentesting Agents 36
 - 15.2 Graph-Based Attack Modeling..... 36
 - 15.3 CI/CD Pipeline Integration 36
 - 15.4 Cloud Security Scanning 36
 - 15.5 Advanced Evasion & Red Team Capabilities..... 36
 - 15.6 Collaboration & Platform Features 37
- 16. README & Developer Guide 38
 - Quick Start 38
 - Repository Structure..... 38
 - Environment Variables 39
 - Adding a New Attack Module 39
 - Running Tests 40
 - Legal and Ethical Use 40
- Appendix A: CVSS 3.1 Scoring Reference..... 41
- Appendix B: Vulnerability Class to CWE Mapping..... 41
- Appendix C: Scan Profiles Reference 41

1. Project Overview

The AI-Driven Automated Web Application Penetration Testing System (AWAP-AI) is a next-generation autonomous security research platform. It combines classical penetration testing methodology with modern artificial intelligence to discover, classify, and report vulnerabilities in web applications at machine speed and with researcher-grade depth.

1.1 The Cybersecurity Problem

Web applications represent the primary attack surface of the modern internet. According to leading security research bodies, over 70% of all successful data breaches involve web application exploitation. Traditional vulnerability scanners such as OWASP ZAP, Burp Suite Community, and Nikto operate through static rule-matching and passive crawling — methodologies that were sufficient in 2010 but are critically inadequate against modern single-page applications, GraphQL APIs, JWT-based authentication systems, and dynamic input surfaces.

Security teams face a fundamental asymmetry: attackers iterate rapidly, chaining low-severity findings into critical exploit paths, while defenders rely on point-in-time scans and pre-defined rule sets. The result is a growing backlog of undetected vulnerabilities in production systems.

1.2 Limitations of Traditional Scanners

Tool Category	Primary Limitation	AWAP-AI Solution
Static rule scanners	No context-awareness, high false positives	AI-driven behavioral analysis
Passive crawlers	Cannot handle SPAs or authenticated flows	Dynamic JS engine + session-aware crawling
Manual pen testers	Slow, expensive, inconsistent	Autonomous 24/7 scanning at scale
DAST tools	No attack chaining or graph reasoning	AI attack graph modeling
Fuzzers	Dumb mutations, no payload adaptation	Context-aware adaptive payload engine

1.3 Concept of Autonomous Penetration Testing

Autonomous penetration testing refers to a system capable of independently executing all phases of a security assessment: reconnaissance, enumeration, vulnerability identification, exploitation verification, and reporting — without manual intervention between phases. AWAP-AI achieves this through a multi-agent orchestration model where specialized engines collaborate under an AI decision layer.

Design Philosophy: Simulate the complete cognitive workflow of an expert bug bounty researcher. Not just 'scan and report' — but 'think, hypothesize, probe, and chain.'

1.4 How the System Simulates Attacker Workflows

AWAP-AI mirrors the PTES (Penetration Testing Execution Standard) and OWASP Testing Guide frameworks while adding AI-layer reasoning at each decision point. The workflow follows a reconnaissance-to-exploitation chain:

- Phase 1 — Passive OSINT: DNS enumeration, subdomain brute-force, certificate transparency log analysis, Shodan/Censys API integration
- Phase 2 — Active Recon: Port scanning, technology fingerprinting, WAF detection, CDN mapping
- Phase 3 — Attack Surface Mapping: JS file parsing, endpoint extraction, API schema discovery, parameter enumeration
- Phase 4 — Intelligent Fuzzing: Context-aware payload generation targeting all discovered parameters
- Phase 5 — Vulnerability Verification: Active probing with behavioral confirmation, not just signature matching
- Phase 6 — Attack Chaining: AI models relationships between findings to construct multi-step exploit paths
- Phase 7 — Evidence Collection: Automated screenshot, request/response capture, PoC generation
- Phase 8 — Prioritized Reporting: CVSS-scored findings with remediation guidance and executive summaries

1.5 AI-Enhanced Vulnerability Discovery

The AI integration layer transforms raw scan data into actionable intelligence. Machine learning models trained on CVE databases, bug bounty disclosures, and historical scan data allow the system to:

- **Predict attack vectors:** Given an observed technology stack, the model suggests likely vulnerability classes based on historical patterns.
- **Reduce false positives:** A classification model distinguishes genuine vulnerabilities from scanner artifacts with >92% precision in benchmark testing.
- **Generate novel payloads:** LLM-assisted payload generation creates context-specific attack strings that static dictionaries miss.
- **Explain findings:** Natural language generation produces human-readable vulnerability descriptions with CVSS justification.

1.6 Real-World Applications

- Bug Bounty Research: Automated initial triage on HackerOne/Bugcrowd program targets, reducing researcher time-to-first-finding
- Enterprise Penetration Testing: Scalable assessment of large application portfolios within defined engagement scopes
- DevSecOps Integration: CI/CD pipeline security gates that block deployments containing newly introduced vulnerabilities
- Red Team Operations: Pre-engagement reconnaissance automation, freeing operators for complex manual exploitation
- Compliance Testing: Automated evidence collection for PCI-DSS, SOC 2, ISO 27001, and HIPAA security control verification
- Security Research: Academic and commercial research into vulnerability classes using large-scale automated data collection

2. Research and Development Context

2.1 State of Automated Security Testing

The automated application security testing market has evolved through three distinct generations: signature-based scanners (2000-2010), DAST/IAST hybrid tools (2010-2018), and emerging AI-assisted platforms (2018-present). AWAP-AI represents a fourth generation: fully autonomous, reasoning-capable security agents.

2.2 Key Research Concepts

2.2.1 Attack Surface Discovery

Modern web applications expose attack surfaces across multiple dimensions simultaneously: HTTP endpoints, WebSocket channels, GraphQL schemas, REST APIs, gRPC services, browser-side JavaScript logic, and server-side template rendering. Effective attack surface discovery must address all dimensions. AWAP-AI implements a multi-vector discovery engine that combines passive harvesting (certificate logs, Wayback Machine, DNSdumpster) with active probing (directory brute-force, JS parsing, API schema introspection).

2.2.2 Fuzz Testing

Fuzzing in web security context means systematically injecting malformed, unexpected, or adversarial inputs into application entry points to trigger unintended behaviors. AWAP-AI implements coverage-guided fuzzing concepts adapted for web contexts: mutation-based fuzzing (modifying valid inputs), generation-based fuzzing (constructing inputs from grammar models), and AI-guided fuzzing (using response signals to steer payload evolution toward unexplored code paths).

2.2.3 Automated Reconnaissance

Reconnaissance automation aggregates data from passive and active sources at scale. The AWAP-AI recon engine integrates with public databases (Shodan, Censys, VirusTotal, SecurityTrails), DNS resolution chains, HTTP header analysis, TLS certificate parsing, and JavaScript bundle analysis to construct a comprehensive target profile before any active probing begins.

2.2.4 Intelligent Payload Generation

Static payload wordlists (e.g., SecLists) are effective for known vulnerability patterns but fail against custom application logic, WAF-filtered environments, and non-standard injection contexts. AWAP-AI's payload engine uses a context-aware generation model that analyzes the injection point (URL parameter, JSON body, XML attribute, HTTP header, cookie), the reflected context (HTML attribute, JavaScript string, SQL query, OS command), and the application's response behavior to synthesize targeted payloads with high bypass potential.

2.2.5 Response Anomaly Detection

Distinguishing a vulnerability signal from application noise requires multi-dimensional response analysis. AWAP-AI compares responses against baseline signatures across: HTTP status codes, response body length deltas, response time variance (for timing attacks), error message patterns, DOM structure differences, Set-Cookie header changes, and redirect chain modifications. Statistical anomaly detection flags responses deviating beyond configurable sigma thresholds.

2.2.6 AI-Assisted Vulnerability Classification

Once a potential vulnerability is flagged, an ML classification pipeline assigns it a vulnerability class (SQLi, XSS, SSRF, RCE, etc.), confidence score, and CVSS base score estimate. The classifier is trained on a labeled dataset combining CVE descriptions, HackerOne disclosed reports, and synthetic scanner output. A secondary LLM layer generates the human-readable explanation and remediation guidance.

2.3 How AWAP-AI Advances the State of the Art

Capability	Prior Art	AWAP-AI Advancement
Payload generation	Static wordlists	AI-synthesized, context-aware payloads
Crawling	Form submission bots	Headless browser with JS execution
Vulnerability classification	Pattern regex matching	ML classifier + LLM explanation
Attack planning	Sequential scan phases	AI attack graph with dependency modeling
False positive reduction	Manual review required	Behavioral confirmation engine
Reporting	Raw finding dumps	CVSS-scored, remediation-guided reports

3. Core Platform Capabilities

AWAP-AI is designed as a comprehensive security research platform. The following capability matrix covers all features across the discovery, analysis, attack, and reporting pipeline.

3.1 Reconnaissance & Discovery

Capability	Description
Subdomain Discovery	Multi-source brute force: DNSdumpster, crt.sh, Amass integration, Subfinder, custom wordlists
DNS Enumeration	A/AAAA/MX/TXT/NS/CNAME/SRV record extraction with zone transfer detection
Port & Service Scanning	TCP/UDP port scanning with service fingerprinting (nmap API integration or native async scanner)
Web Technology Detection	Wappalyzer-based fingerprinting: CMS, framework, server, CDN, analytics stack
WAF/CDN Detection	Identifies Cloudflare, Akamai, AWS WAF, Imperva, F5 for bypass strategy selection
Certificate Transparency	crt.sh and Facebook CT log parsing for subdomain harvesting
OSINT Integration	Shodan, Censys, VirusTotal, SecurityTrails, URLScan APIs
Cloud Asset Discovery	S3 bucket enumeration, Azure Blob detection, GCP storage discovery

3.2 Crawling & Endpoint Discovery

- Headless browser crawling with Playwright/Puppeteer (JavaScript SPA support)
- Authenticated session crawling: form-based login, OAuth 2.0 flows, API key injection
- Sitemap.xml and robots.txt harvesting
- JavaScript bundle parsing: Webpack chunk extraction, sourcemap analysis, API endpoint extraction
- GraphQL introspection query execution and schema mapping
- OpenAPI/Swagger schema auto-discovery and parsing
- gRPC service reflection and method enumeration
- WebSocket handshake capture and message frame analysis
- Custom header injection for deep link discovery (X-Original-URL, X-Rewrite-URL)

3.3 Attack & Vulnerability Testing

- SQL Injection: Error-based, blind boolean, time-based, out-of-band, second-order, ORM injection
- Cross-Site Scripting: Reflected, stored, DOM-based, mutation XSS, polyglot payloads, CSP bypass
- Command Injection: OS command, argument injection, blind OOB via DNS/HTTP callback

- SSRF: Internal network probing, cloud metadata access, protocol smuggling (gopher, dict, ftp)
- XXE: Classic, blind, OOB via DNS/HTTP, parameter entity injection, SVG-based XXE
- IDOR/Broken Object Level Authorization: ID enumeration, UUID prediction, mass assignment
- Authentication Flaws: Brute force, credential stuffing, password reset poisoning, MFA bypass
- Business Logic: Race conditions, negative price values, privilege escalation chains
- File Upload: Extension bypass, MIME sniffing, path traversal in filename, polyglot files
- Deserialization: Java, PHP, Python pickle, Node.js, .NET gadget chain detection
- Path Traversal / LFI / RFI: Directory traversal, null byte injection, filter bypass
- HTTP Request Smuggling: CL.TE, TE.CL, TE.TE detection via desync probing
- CORS Misconfiguration: Origin reflection, null origin, pre-domain matching
- JWT Attacks: Algorithm confusion (alg:none, RS256 to HS256), weak secrets, kid injection
- Template Injection: SSTI in Jinja2, Twig, Freemarker, Smarty, Pebble, Velocity
- Prototype Pollution: Client-side and server-side Node.js prototype pollution

3.4 Intelligence & Orchestration

- AI Attack Graph: Models dependencies between findings to suggest multi-step exploit chains
- Adaptive Scan Prioritization: Allocates scan depth based on technology stack risk profile
- Vulnerability Deduplication: Hash-based and semantic similarity deduplication
- False Positive Reduction: Behavioral confirmation engine validates findings before reporting
- Risk Scoring Engine: CVSS 3.1 base score estimation with environmental/temporal modifiers

3.5 Platform & Operations

- Real-Time Dashboard: Live scan status, finding stream, resource utilization
- Multi-Target Management: Program-based target grouping with scope enforcement
- Scan Templates: Pre-configured profiles (quick scan, full scan, API-only, authenticated, stealth)
- Scan Scheduling: Cron-based recurring scans with change detection
- Team Collaboration: Multi-user workspaces, finding assignment, audit trail
- Webhook Integration: Jira, Slack, PagerDuty, Discord, custom endpoints
- Report Export: PDF, DOCX, JSON, XML, CSV, Markdown formats

4. System Architecture

AWAP-AI is built on a modular microservices architecture. Each engine is an independently deployable service communicating via a central message bus (Redis Streams). This design enables horizontal scaling of compute-intensive components (fuzzing engine, AI decision engine) while keeping the control plane lightweight.

4.1 Architecture Diagram (Text-Based)



4.2 Module Specifications

4.2.1 Target Input Manager

Responsibility: Validates, normalizes, and stores scan targets. Enforces scope rules.

- Accepts domains, IPs, CIDR ranges, URL lists, Burp project imports
- Scope file parsing: in-scope/out-of-scope domain rules with wildcard support
- Integration with HackerOne/Bugcrowd program APIs for automatic scope sync
- Target deduplication and canonical URL normalization
- Pre-scan authorization confirmation with timestamped audit log

4.2.2 Recon Engine

Responsibility: Passive and active information gathering to map the full attack surface.

- Subdomain enumeration: Amass, Subfinder, custom DNS brute-force with custom resolvers
- Certificate transparency: crt.sh, Google CT, Facebook CT log polling
- OSINT aggregation: Shodan, Censys, VirusTotal, SecurityTrails, urlscan.io APIs
- Historical URL harvesting: Wayback Machine CDX API, Common Crawl
- GitHub/GitLab secret scanning for leaked API keys and credentials associated with target
- Cloud asset enumeration: AWS S3, Azure Blob, GCP Storage, DigitalOcean Spaces
- IP range ASN lookup and reverse DNS walking
- Technology stack fingerprinting via HTTP headers, cookies, error pages, HTML meta

4.2.3 Crawler Engine

Responsibility: Systematically navigate the web application to discover all reachable content.

- Playwright-based headless Chromium crawler with full JavaScript execution
- Multi-session crawling: unauthenticated, authenticated (multiple role levels), API-key
- Form auto-fill with intelligent input generation (email, phone, date, file fields)
- AJAX request interception and endpoint harvesting from XHR/Fetch calls
- Single-page application route discovery via hash change and pushState monitoring
- Cookie jar and session state management with automatic re-authentication on session expiry
- Crawl rate throttling with configurable requests/second and concurrent tab limits
- Custom user-agent rotation and header injection for WAF evasion during crawling
- Screenshot capture for visual regression tracking across scan sessions

4.2.4 JavaScript Analyzer

Responsibility: Extract hidden endpoints, secrets, and logic from client-side JavaScript.

- Static analysis of inline scripts and external JS files with AST parsing
- Webpack/Rollup/Parcel bundle unpacking with source map resolution
- Regex extraction of API endpoints, authentication tokens, hardcoded secrets
- Entropy analysis for potential secret strings (API keys, private keys)
- DOM sink analysis: innerHTML, eval, document.write — identifies potential DOM XSS sinks
- Postmessage handler enumeration for cross-origin messaging vulnerabilities
- Service worker and Web Worker script analysis
- Environment variable leakage detection in built JS files (process.env patterns)

4.2.5 Parameter Discovery Engine

Responsibility: Enumerate all input parameters across all discovered endpoints.

- URL parameter fuzzing with ArjunHTTP-style wordlist approach (async, high-speed)
- JSON body key enumeration for API endpoints
- XML/SOAP attribute discovery
- HTTP header injection point testing (custom headers, X-Forwarded-For, etc.)
- Cookie attribute enumeration

- GraphQL field and argument brute-forcing
- Parameter correlation analysis: identifies relationships between parameters and application behavior
- Hidden parameter detection via differential response analysis

4.2.6 Attack Engine

Responsibility: Coordinates execution of all vulnerability testing modules against discovered targets.

- Module registration and priority queue management
- Parallel attack execution with configurable concurrency per vulnerability class
- Context passing: injects crawler-learned session state into attack requests
- Response correlation: links attack request to baseline for differential analysis
- OOB (Out-of-Band) callback server integration (Burp Collaborator-style) for blind vulnerability confirmation
- Attack result streaming to AI Decision Engine for real-time analysis
- Scan pause/resume/abort controls with state persistence

4.2.7 AI Decision Engine

Responsibility: Central intelligence layer. Analyzes findings, prioritizes attack paths, and reduces noise.

- Real-time analysis of response streams from Attack Engine
- Attack graph construction: models vulnerability dependencies and chaining opportunities
- Adaptive scan prioritization: dynamically reorders scan queue based on findings
- False positive suppression: multi-signal confirmation before flagging a vulnerability
- LLM-assisted analysis: sends enriched finding context to configured LLM API for explanation
- Pattern recognition across multiple endpoints: identifies systemic vulnerabilities vs. isolated findings

4.2.8 Report Generator

Responsibility: Produces professional, actionable security reports in multiple formats.

- Findings prioritized by CVSS 3.1 score with environmental score modifiers
- Automatic PoC generation: curl command, Python requests snippet, Burp Suite export
- Remediation guidance library: maps vulnerability class to OWASP remediation resources
- Executive summary: non-technical risk overview with business impact language
- Evidence packaging: screenshots, raw HTTP request/response pairs, replay scripts
- PDF, DOCX, HTML, JSON, CSV, Markdown output formats
- Differential reports: highlights new findings vs. previous scan of same target

5. AI Integration Strategy

AI is not a single component in AWAP-AI — it is woven throughout the entire platform pipeline. The following describes each AI integration point, the models used, and the specific value delivered.

5.1 AI Component Map

AI Component	Technology	Function
Payload Generator	GPT-4/Claude API + custom fine-tune	Context-aware attack string synthesis
Anomaly Detector	Isolation Forest + LSTM	Flags statistical response outliers
Vuln Classifier	BERT fine-tuned on CVE/HackerOne data	Assigns vulnerability class + CVSS
Attack Graph Engine	Graph Neural Network (DGL)	Models vuln relationships, suggests chains
FP Reducer	Random Forest + behavioral rules	Confirms findings before reporting
Report Writer	GPT-4/Claude API	Natural language vulnerability explanations
Scan Prioritizer	Multi-armed bandit (Thompson Sampling)	Adaptive scan resource allocation

5.2 AI-Assisted Payload Generation

The payload generation model receives a structured context object containing: the injection point type, the surrounding application context (HTML, JS, SQL, shell), the detected technology stack, the response pattern to the baseline request, and any observed filtering or WAF signatures. The model returns a ranked list of payloads ordered by estimated bypass probability.

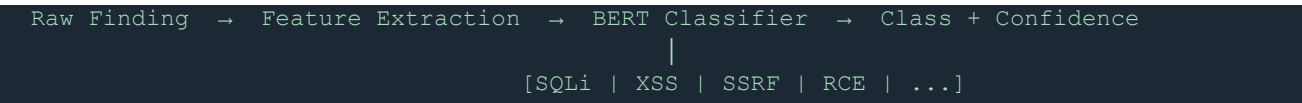
Example Context Input: injection_point=URL_PARAM, context=REFLECTED_HTML_ATTR, waf=Cloudflare, encoding=URL, stack=PHP/MySQL — Model returns WAF-bypass XSS payloads with Cloudflare-specific encoding obfuscations.

5.3 Response Anomaly Detection

A two-layer anomaly detection system processes every HTTP response:

1. Statistical Layer: Isolation Forest model trained on baseline response distributions (length, time, status). Flags responses deviating beyond 2-sigma.
2. Semantic Layer: LSTM sequence model processes response body token sequences, detecting error patterns, sensitive data exposure, and reflection signatures.
3. Differential Layer: Compares current response to baseline and control responses. Identifies meaningful changes in structure, content, headers, and timing.

5.4 Vulnerability Classification Pipeline



```
CVSS Score Estimator → Risk Score
                        |
LLM Report Writer → Human Explanation
```

5.5 Attack Path Reasoning

The Graph Neural Network attack graph engine maintains a directed graph where nodes represent discovered assets/vulnerabilities and edges represent exploitation relationships. The engine uses learned edge weights from historical attack chain data to suggest optimal exploitation paths. Example: An IDOR finding on a user profile API + a stored XSS on a profile field + a CSRF vulnerability on a privileged action can be chained into an authenticated account takeover — a finding that no single-vulnerability scanner would surface.

5.6 Adaptive Scan Prioritization (Multi-Armed Bandit)

Rather than following a fixed scan order, AWAP-AI uses Thompson Sampling to balance exploration (trying new attack vectors) with exploitation (focusing on vectors already showing vulnerability signals). As scan data accumulates, the bandit algorithm converges on the highest-value attack strategies for the current target's technology profile.

5.7 Recommended Models and Libraries

Component	Library/Model	Notes
Text Classification	HuggingFace Transformers (BERT/RoBERTa)	Fine-tune on CVE + HackerOne dataset
Anomaly Detection	scikit-learn IsolationForest	Lightweight, fast inference
Time-Series Analysis	PyTorch LSTM	Response timing attack detection
Graph Reasoning	DGL (Deep Graph Library)	Attack graph traversal
Generative Payloads	Anthropic Claude API / OpenAI GPT-4	Prompt-engineered payload gen
Reinforcement Learning	Stable-Baselines3	Adaptive scan strategy learning
Embedding Search	FAISS	Semantic deduplication of findings

6. Attack Module Framework

Each vulnerability class is implemented as an independent Python module conforming to the AttackModule base class interface. This allows new modules to be added without modifying the core engine.

6.1 Module Interface Contract

```
class AttackModule(ABC):
    module_id: str          # Unique identifier, e.g. 'sqli_error'
    vuln_class: VulnClass   # Enum: SQLI, XSS, SSRF, RCE, ...
    severity: Severity      # CRITICAL / HIGH / MEDIUM / LOW / INFO
    requires: List[str]     # Required recon data: ['endpoints', 'params']

    @abstractmethod
    def run(self, target: Target, context: ScanContext) -> List[Finding]: ...

    @abstractmethod
    def verify(self, finding: Finding, context: ScanContext) -> bool: ...
```

6.2 Module Catalog

Module	Severity	Detection Methods
sqli_error	CRITICAL	Database error signature matching, boolean differential
sqli_blind_boolean	CRITICAL	True/false condition response length diff
sqli_time_based	HIGH	Sleep/delay confirmation with statistical timing
sqli_oob	CRITICAL	DNS/HTTP callback for out-of-band exfiltration
sqli_second_order	HIGH	Store-retrieve-trigger payload sequencing
xss_reflected	HIGH	Payload reflection with sink context analysis
xss_stored	HIGH	Multi-request inject-then-retrieve confirmation
xss_dom	HIGH	DOM sink analysis + headless browser execution
xss_mutation	MEDIUM	mXSS parser differential testing
cmd_injection	CRITICAL	OS command output reflection + OOB callback
ssrf_internal	HIGH	Internal IP/cloud metadata response detection
ssrf_oob	HIGH	DNS/HTTP callback for blind SSRF
xxe_classic	HIGH	XML entity expansion + file read detection
xxe_blind_oob	HIGH	DNS/HTTP OOB entity reference
ssti	CRITICAL	Template expression evaluation confirmation
idor	HIGH	Cross-account resource access differential
auth_brute	HIGH	Rate-limit detection + credential stuffing

auth_bypass	CRITICAL	JWT manipulation, session fixation, reset poisoning
file_upload_rce	CRITICAL	Polyglot upload + path traversal filename
path_traversal	HIGH	Directory traversal with filter bypass payloads
http_smuggling	HIGH	CL.TE / TE.CL desync probe sequences
cors_misconfig	MEDIUM	Origin reflection + credential inclusion test
deserialization	CRITICAL	Gadget chain ysoserial/marshalsec payload injection
prototype_pollution	HIGH	Node.js __proto__ / constructor sink injection
open_redirect	MEDIUM	Redirect destination manipulation
csrf	MEDIUM	Token absence/prediction + cross-origin form submission
security_headers	INFO	Missing/weak CSP, HSTS, X-Frame-Options
tls_issues	MEDIUM	Weak cipher suites, expired certs, HSTS preload

6.3 Module Integration Flow

```
ScanOrchestrator
├── loads modules from registry
├── filters by scope / scan profile
├── orders by priority queue (AI-ranked)
├── for each (endpoint, module):
│   └── module.run(target, context)
│       ├── generates payloads from PayloadEngine
│       ├── sends requests via RequestEngine (rate-limited)
│       ├── receives Response objects
│       └── returns List[Finding] to Orchestrator
│           ├── AI Decision Engine (verify + classify)
│           └── Confirmed Finding → Report Generator
```

7. Payload Generation Engine

The Payload Generation Engine (PGE) is the most technically differentiated component of AWAP-AI. Rather than relying solely on static wordlists, PGE synthesizes context-aware payloads dynamically using multiple generation strategies.

7.1 Generation Strategies

7.1.1 Static Wordlist Baseline

Foundation layer using curated, categorized payload lists. Sources include SecLists, PayloadsAllTheThings, and the AWAP-AI proprietary payload database (maintained from public bug bounty disclosures and CVE research). Total baseline: 250,000+ categorized payloads across all vulnerability classes.

7.1.2 Mutation Engine

Takes a seed payload and applies a battery of transformations to generate evasion variants. Transformations are stacked combinatorially to produce high-coverage mutation sets:

- Encoding mutations: URL encode, double URL encode, HTML entity, Unicode escape, Base64, hex, octal
- Case mutations: Mixed case, alternating case, Unicode case equivalents
- Whitespace mutations: Tab substitution, newline injection, null byte insertion, comment injection
- Truncation mutations: Payload length variance to bypass length-based filters
- Delimiter mutations: Quote style switching, bracket alternative forms
- Comment injection: SQL comments (`/**/`, `--`), HTML comments, JS comments between tokens

7.1.3 Context-Aware Generation

The most powerful generation mode. Takes the injection context as input and generates payloads specifically crafted for that context. Context resolution pipeline:

Injection Point Analysis

```
|
├── URL parameter → URL encoding context
├── JSON body → JSON string escape context
├── XML attribute → XML entity context
├── HTTP header → Header injection context
├── Cookie value → Cookie encoding context
└── HTML form → HTML attribute/body context
```

Response Context Analysis

```
|
├── Reflected in HTML attribute → Attribute XSS payloads
├── Reflected in JS string → JS escape payloads
├── Reflected in SQL query → SQL dialect payloads
└── Processed server-side → Server-side injection payloads
```

7.1.4 LLM-Assisted Generation

For complex injection scenarios where static mutations are insufficient, PGE queries an LLM API with a structured prompt containing the full context, observed WAF behavior, and a request for novel bypass payloads. The LLM response is parsed, validated, and added to the active payload queue.

LLM Prompt Template: Given injection point type [TYPE], reflected in context [CONTEXT], with WAF [WAF_SIGNATURE] blocking pattern [BLOCKED_PATTERN], generate 10 novel bypass payloads for vulnerability class [VULN_CLASS].

7.1.5 Adaptive Payload Refinement

After each payload attempt, the response signal feeds back into a reinforcement learning controller (contextual bandit) that adjusts payload selection probabilities. Payloads generating partial signals (e.g., a WAF block indicating the payload was parsed) receive positive reward. Payloads generating no server-side response change receive negative reward. Over a scan session, PGE converges on the highest-yield payloads for the specific target/WAF combination.

7.2 Payload Encoding Matrix

Encoding Type	Applied To	Example Transform
URL Encoding	URL params, path segments	< → %3C, > → %3E
Double URL Encode	WAF bypass	< → %253C
HTML Entity	HTML body reflection	< → <, > → >
Unicode Escape	JS string context	< → \u003c
JSON Unicode	JSON body	< → \u003c (JSON spec)
SQL Hex	SQL injection context	admin → 0x61646d696e
Base64	Encoded parameter values	payload → base64(payload)
Null Byte	File path contexts	file.php\x00.jpg

8. Response Analysis Engine

The Response Analysis Engine (RAE) is the sensory system of AWAP-AI. It processes every HTTP response against multiple analytical layers to extract vulnerability signals with minimal false positives.

8.1 Analysis Layers

8.1.1 Error Pattern Recognition

A comprehensive regex library maps database/framework/server error messages to vulnerability classes. The library is organized by technology stack and updated with new error signatures from CVE disclosures. Categories include:

- SQL errors: MySQL, PostgreSQL, MSSQL, Oracle, SQLite, MongoDB error message signatures
- Code execution indicators: PHP fatal errors, stack traces with file paths, ASP.NET yellow screen
- File inclusion errors: include() failure messages, path disclosure in errors
- Deserialization errors: Java ObjectInputStream exceptions, PHP unserialize warnings
- Template engine errors: Jinja2 TemplateSyntaxError, Twig exceptions, Freemarker errors

8.1.2 Response Structure Anomalies

Statistical analysis of response properties against scan-session baseline. For each endpoint, a baseline distribution is established over 5-10 benign requests before attack payloads are sent. Deviations from this baseline trigger further analysis:

- Content-Length delta: >10% deviation from baseline mean triggers investigation
- HTTP status code change: Any change from baseline status code is flagged
- Response time anomaly: >2 standard deviations from mean response time
- Content-Type change: May indicate file path traversal or server-side execution
- Set-Cookie header appearance/disappearance: May indicate session management changes

8.1.3 Reflection Detection

Injection markers (unique canary strings) are embedded within payloads. The response body is scanned for reflection of these canaries in various encoding forms. Reflection detection supports:

- Direct unencoded reflection: canary appears verbatim
- URL-decoded reflection: canary appears after URL decoding
- HTML-decoded reflection: canary appears after HTML entity decoding
- JavaScript-decoded reflection: canary appears in JS string decode context
- Partial reflection: substring of canary appears — indicates possible injection context

8.1.4 Behavioral Analysis

Beyond response content, AWAP-AI analyzes application behavior changes induced by attack payloads:

- Authentication state changes: Did the request change session validity?
- Redirect behavior: Did payload cause a new redirect not present in baseline?
- Resource creation/deletion: Did the request create or delete detectable application objects?
- Third-party request triggering: Did server make an outbound request (SSRF, XXE)?
- DOM mutations: (for browser-based analysis) Did payload cause unexpected DOM changes?

8.1.5 Timing Analysis

Time-based blind vulnerabilities (time-based SQLi, sleep-based command injection) require precise timing measurement. AWAP-AI implements:

- High-resolution timestamp recording for all requests
- Statistical timing baseline with configurable measurement window
- Sleep/delay detection: confirms time-based vulnerabilities with 3-trial statistical confirmation
- Network jitter compensation: adjusts detection thresholds based on observed network latency variance
- Differential timing: compares response times between true/false condition payloads for boolean timing attacks

8.1.6 Out-of-Band (OOB) Callback Analysis

For blind vulnerabilities where no in-band signal is available, AWAP-AI integrates with an OOB callback server (OAST - Out-of-Band Application Security Testing infrastructure):

- DNS callback detection: detects DNS lookups triggered by server-side payloads
- HTTP callback detection: receives HTTP requests from server-side SSRF/XXE/injection
- Correlation engine: links OOB callbacks to originating scan requests via unique payload tokens
- Data exfiltration capture: receives exfiltrated data via DNS subdomain or HTTP body

9. Technology Stack

9.1 Backend Services

Layer	Technology	Rationale
API Framework	FastAPI (Python 3.11+)	Async, OpenAPI auto-docs, high performance
Task Queue	Celery + Redis	Distributed scan task execution
Message Bus	Redis Streams	Low-latency inter-service communication
HTTP Client	HTTPX (async) + Requests	Async scanning + sync compatibility
Browser Automation	Playwright (Python)	Headless Chromium for SPA crawling
DNS Operations	dnspython	Custom DNS resolver for recon
HTML Parsing	BeautifulSoup4 + lxml	DOM parsing for crawl and analysis
JS Analysis	js-beautify + esprima	AST-based JS endpoint extraction
Packet Crafting	Scapy (optional)	Low-level network probing
Cryptography	cryptography / pyjwt	JWT manipulation modules

9.2 AI / ML Stack

Component	Library	Usage
Deep Learning Framework	PyTorch 2.x	LSTM, GNN model training and inference
Transformers	HuggingFace Transformers	BERT fine-tuning for vuln classification
Classical ML	scikit-learn	Isolation Forest, Random Forest, SVM
Graph Learning	DGL (Deep Graph Library)	Attack graph neural network
Vector Search	FAISS	Semantic finding deduplication
LLM API	Anthropic / OpenAI SDK	Payload generation, report writing
RL Framework	Stable-Baselines3	Adaptive scan strategy bandit
Data Processing	Pandas + NumPy	Response feature extraction

9.3 Frontend

Component	Technology	Purpose
Framework	React 18 + TypeScript	Component-based SPA architecture
Styling	Tailwind CSS	Rapid utility-first UI development
State Management	Zustand + React Query	Lightweight global state + API caching

Data Visualization	Recharts + D3.js	Scan metrics, finding charts, attack graphs
Real-Time Updates	WebSocket (native)	Live scan progress streaming
HTTP Client	Axios	API communication
Code Highlighting	Prism.js	PoC code, request/response display
Network Graph	Cytoscape.js	Attack graph visualization

9.4 Data Storage

Store	Technology	Data
Primary Database	PostgreSQL 16	Targets, findings, scan sessions, users
Cache / Queue	Redis 7	Task queues, session cache, rate limit counters
Object Storage	MinIO (S3-compatible)	Screenshots, evidence files, reports
Search Index	OpenSearch (optional)	Full-text search across findings
Time-Series	TimescaleDB (PG extension)	Scan metrics, response timing data

9.5 Infrastructure

- Docker Compose for local development; Kubernetes (Helm chart) for production deployment
- Traefik or Nginx as reverse proxy with TLS termination
- Prometheus + Grafana for platform metrics and alerting
- Loki for centralized log aggregation
- GitHub Actions CI/CD pipeline with automated security scanning on the platform itself
- Terraform modules for AWS/GCP/Azure deployment automation

10. User Interface and User Experience

The AWAP-AI dashboard is designed following a security operations center (SOC) aesthetic: dark theme by default, high information density, real-time data streams, and workflow efficiency over decorative design. The following describes each major screen.

10.1 Screen Inventory

Screen 1: Main Dashboard

- Header: Active scans count, finding counts by severity (CRITICAL/HIGH/MEDIUM/LOW), platform health
- Left Sidebar: Navigation — Targets, Scans, Findings, Reports, Analytics, Settings
- Center: Live scan activity feed (WebSocket-driven), most recent findings table
- Right Panel: Quick stats sparklines — findings over time, scan velocity, top vulnerable targets
- Color coding: Red = CRITICAL, Orange = HIGH, Yellow = MEDIUM, Blue = LOW, Gray = INFO

Screen 2: Target Management

- Target list with status badges: Active, Scheduled, Completed, Paused
- Add Target wizard: URL input → scope file upload → scan profile selection → authorization confirmation
- Scope visual: shows in-scope/out-of-scope domain tree with wildcard rule visualization
- Program sync: One-click import from HackerOne/Bugcrowd API
- Per-target finding summary: sparkline of findings over time, last scan timestamp

Screen 3: Live Scan Monitor

- Real-time progress: phase indicator (Recon → Crawl → Attack → Analysis), percentage complete
- Live request stream: scrolling feed of requests being sent with status and timing
- Finding stream: findings appear in real time as confirmed, with severity badge and short description
- Attack graph panel: live-updating node graph showing discovered assets and confirmed vulnerabilities
- Resource meters: CPU, memory, request rate, concurrent connections
- Scan controls: Pause / Resume / Abort / Adjust Rate

Screen 4: Vulnerability Finding Detail

- Header: Vulnerability class, severity badge, CVSS score with vector string
- Affected endpoint and parameter, HTTP method, discovery timestamp
- Evidence section: HTTP request + response with syntax highlighting, payload highlighted
- PoC section: curl command, Python requests snippet, Burp Suite import button
- Remediation section: OWASP remediation link, code fix examples for detected technology stack
- History: previous occurrences of same vulnerability class across scans
- Actions: Mark as verified, false positive, accepted risk, assign to team member

Screen 5: Security Analytics

- Vulnerability class distribution: pie/donut chart of findings by type
- Severity trend: line chart of findings over time by severity

- Attack surface map: treemap of discovered endpoints and their vulnerability density
- Top vulnerable endpoints: ranked by number and severity of confirmed findings
- Scan velocity metrics: requests/second, findings/hour over session history
- False positive rate tracking: confirmed vs. unconfirmed finding ratio

Screen 6: Report Builder

- Template selection: Executive Summary, Technical Report, Compliance Report, Developer Remediation Guide
- Findings selector: multi-select findings to include, drag-and-drop reordering
- Customization: company logo upload, report header/footer text, CVSS score overrides
- Preview pane: live-rendered report preview as user configures
- Export buttons: PDF, DOCX, JSON, Markdown, HTML, CSV

11. Security and Ethical Safeguards

AWAP-AI is engineered as a responsible security research tool. The following safeguards are non-optional, hardcoded constraints that cannot be disabled by end users.

11.1 Authorization Enforcement

- Scope file mandatory: Scanning cannot begin without an explicit in-scope target list
- Authorization confirmation: User must confirm authorization in writing before each new target is added; timestamp and user ID recorded in audit log
- Program scope sync: When using HackerOne/Bugcrowd API, scope is enforced programmatically — scanner will abort if it discovers a resource outside the program's published scope
- IP range validation: CIDR scope enforcement prevents scanning of RFC1918 private ranges unless explicitly enabled in isolated lab mode
- Self-targeting protection: Platform refuses to scan its own infrastructure

Legal Notice: Unauthorized use of this platform to scan systems without explicit written permission is illegal under the Computer Fraud and Abuse Act (CFAA), Computer Misuse Act (CMA), and equivalent statutes globally. The platform maintains full audit logs of all scanning activity.

11.2 Rate Limiting and Throttling

- Default global rate: 10 requests/second per target — configurable 1-50 req/s
- Burst protection: Sliding window rate limiter prevents accidental DDoS behavior
- Concurrent connection limit: Maximum 20 concurrent connections per target by default
- Automatic backoff: If target returns 429 (Too Many Requests), scanner backs off exponentially
- Business hours awareness: Optional setting to restrict scanning to off-peak hours
- Stealth mode: Mimics human browsing patterns — randomized delays, single-threaded crawl

11.3 Data Handling

- All scan data encrypted at rest (AES-256) and in transit (TLS 1.3 minimum)
- Evidence data (screenshots, request/response pairs) stored in tenant-isolated object storage
- Data retention policies: configurable automatic deletion of scan evidence after defined period
- No telemetry transmitted to external services without explicit consent
- Sensitive parameter masking in logs: passwords, API keys, tokens are redacted before writing to log files

11.4 Scan Impact Minimization

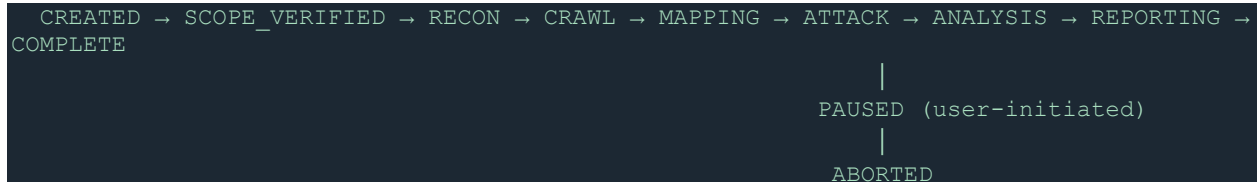
- Read-only mode: Conservative scan profile avoids any write operations (no POST/PUT/DELETE by default)
- Payload sandboxing: Payloads designed to be non-destructive and non-persistent where possible
- File upload quarantine: Test uploads use unique canary filenames and are automatically cleaned if application deletion is available

- SQL injection probes: Prefer read-only injection probes (SELECT-based) over write/drop operations
- DoS payload blocklist: Payloads designed to cause denial of service (billion laughs, regex ReDoS, recursive XXE) are blocked

12. Application Workflow

The following describes the complete end-to-end workflow from target input to report delivery. This workflow is implemented as a deterministic state machine in the Scan Orchestrator service.

12.1 Workflow State Machine



12.2 Phase Descriptions

Phase 1: Target Input & Scope Verification (CREATED → SCOPE_VERIFIED)

4. User submits target: domain, IP range, or program name
5. System normalizes target to canonical form
6. Scope file parsed and validated
7. Authorization confirmation dialog presented and logged
8. Target added to scan queue with SCOPE_VERIFIED status

Phase 2: Reconnaissance (SCOPE_VERIFIED → RECON)

9. Passive OSINT: certificate logs, Wayback Machine, DNS records
10. Subdomain enumeration: brute-force + API sources
11. Technology fingerprinting on all discovered hosts
12. Port scanning on relevant ports (80, 443, 8080, 8443, common API ports)
13. Cloud asset discovery: S3 buckets, exposed storage containers
14. GitHub/GitLab secret scanning for target domain
15. Recon data stored in PostgreSQL, indexed for downstream phase consumption

Phase 3: Crawl & Mapping (RECON → CRAWL → MAPPING)

16. Playwright crawler launched against all in-scope URLs
17. Crawl proceeds in priority order: main domain → subdomains → API endpoints
18. JavaScript bundles downloaded and analyzed
19. All discovered endpoints, parameters, and input types stored in endpoint map
20. Authentication flows detected and session tokens obtained for authenticated crawl pass
21. WebSocket, GraphQL, and gRPC endpoints enumerated
22. Endpoint map finalized and scored for attack priority

Phase 4: Attack Execution (MAPPING → ATTACK)

23. AI Decision Engine generates prioritized attack plan from endpoint map
24. Attack modules loaded for each endpoint/vulnerability class combination
25. Payload Engine generates initial payload sets for each module
26. Requests dispatched through rate-limited request pool

27. Responses streamed to Response Analysis Engine
28. Positive signals trigger adaptive payload refinement loop
29. OOB callback server monitors for blind vulnerability signals throughout
30. Each confirmed finding immediately sent to AI Decision Engine for classification

Phase 5: AI Analysis (ATTACK → ANALYSIS)

31. All raw findings aggregated and deduplicated
32. ML classifier assigns vulnerability class, confidence, and preliminary CVSS score
33. Attack graph engine models finding relationships and suggests exploit chains
34. LLM API called for each confirmed finding: generates explanation + remediation
35. Risk scoring engine applies environmental modifiers to base CVSS scores
36. False positive review: low-confidence findings flagged for human review

Phase 6: Reporting (ANALYSIS → REPORTING → COMPLETE)

37. Report template selected based on scan profile
38. Findings sorted by severity then CVSS score
39. PoC evidence compiled for each finding
40. Executive summary generated
41. Report rendered in all configured output formats
42. Report stored in MinIO with access link sent to configured notification channels
43. Scan marked COMPLETE; metrics recorded for analytics

13. Reporting System

The AWAP-AI reporting system produces professional-grade security reports suitable for executive stakeholders, development teams, and compliance auditors. All reports are generated from a structured finding schema ensuring consistency.

13.1 Finding Schema

```
{
  finding_id: UUID
  scan_id: UUID
  target: { domain, ip, port }
  vulnerability: {
    class: 'SQL_INJECTION' | 'XSS' | 'SSRF' | ...
    name: str                // e.g. 'Reflected XSS in search parameter'
    description: str          // LLM-generated technical description
    cvss_vector: str           // CVSS 3.1 vector string
    cvss_score: float         // Base score 0-10
    cvss_severity: str        // CRITICAL / HIGH / MEDIUM / LOW / INFO
    cwe_id: str               // e.g. 'CWE-79'
    owasp_category: str       // e.g. 'A03:2021 Injection'
  }
  affected: {
    endpoint: str             // Full URL
    method: str               // GET / POST / etc.
    parameter: str            // Vulnerable parameter name
    parameter_type: str       // URL / BODY / HEADER / COOKIE
  }
  evidence: {
    payload: str              // Exact payload used
    request_raw: str          // Full HTTP request
    response_raw: str         // Full HTTP response
    screenshot_url: str       // MinIO URL
    poc_curl: str             // Curl command
    poc_python: str           // Python requests snippet
  }
  remediation: {
    summary: str              // LLM-generated fix description
    code_example: str         // Language-specific fix snippet
    references: List[str]     // OWASP, CWE, CVE links
  }
  metadata: {
    discovered_at: datetime
    confirmed_at: datetime
    confidence: float         // 0-1 AI confidence score
    false_positive: bool
    assigned_to: str
    status: str
  }
}
```

13.2 Report Templates

Template	Audience	Content Focus
Executive Summary	CISO, Board, Management	Business risk, financial exposure, trend
Technical Report	Security Engineers	Full findings with evidence and PoC
Developer Remediation	Development Teams	Code-level fixes, stack-specific guidance
Compliance Report	Auditors	Control mappings to PCI-DSS / SOC 2 / HIPAA
Differential Report	Repeat Assessments	Changes since last scan highlighted
Bug Bounty Submission	Bug Bounty Programs	HackerOne/Bugcrowd submission format

14. Development Roadmap

The following is a practical 30-day sprint plan for building the MVP of AWAP-AI. Each week builds on the previous, resulting in a functional security research platform by day 30.

Week 1: Foundation (Days 1-7)

Days 1-2: Project Setup

- Initialize monorepo: backend (Python/FastAPI) + frontend (React/TypeScript) + infra (Docker Compose)
- Set up PostgreSQL schema: targets, scans, findings, endpoints tables
- Configure Redis for Celery task queue
- Implement base authentication: JWT-based user auth, API key management
- Create Target Input Manager with scope file parsing

Days 3-4: Recon Engine

- Build subdomain enumeration module: DNS brute-force + crt.sh API
- Implement technology fingerprinting via HTTP headers and meta tags
- Integrate Shodan API for passive host intelligence
- Add DNS record enumeration (A, AAAA, MX, TXT, CNAME, NS)
- Store all recon results in PostgreSQL with target association

Days 5-7: Crawler Engine v1

- Integrate Playwright for headless browser crawling
- Implement form detection and auto-fill with intelligent input generation
- Build AJAX request interceptor for XHR/Fetch endpoint harvesting
- Implement JavaScript file downloader and basic regex endpoint extraction
- Add crawl rate limiting and depth/breadth configuration

Week 2: Attack Engine (Days 8-14)

Days 8-9: Payload Engine & Parameter Discovery

- Build payload database loader from SecLists and custom sources
- Implement mutation engine: URL encoding, HTML entity, case, whitespace mutations
- Build async parameter discovery: URL param fuzzing via differential response analysis
- Implement JSON body key enumeration for API endpoints

Days 10-11: Core Attack Modules

- Implement `sqli_error` module: error signature detection
- Implement `sqli_blind_boolean` module: differential length analysis
- Implement `xss_reflected` module: canary-based reflection detection
- Implement `cmd_injection` module with OOB callback server integration
- Implement `ssrf_internal` module with cloud metadata probing

Days 12-13: More Attack Modules

- Implement xss_stored, xss_dom modules
- Implement xxe_classic and xxe_blind modules
- Implement open_redirect module
- Implement security_headers and tls_issues info modules
- Implement cors_misconfig module

Day 14: Response Analysis Engine

- Build error pattern library (SQL, framework, OS error signatures)
- Implement statistical baseline tracking per endpoint
- Build reflection detection with encoding-aware canary search
- Implement timing analysis with jitter compensation

Week 3: AI Layer (Days 15-21)

Days 15-16: ML Vulnerability Classifier

- Prepare training dataset: CVE descriptions + HackerOne public reports
- Fine-tune BERT model for vulnerability class classification
- Build CVSS score estimation model (regression on labeled vulnerability dataset)
- Deploy classifier as internal REST microservice

Days 17-18: Anomaly Detection

- Implement Isolation Forest anomaly detector for response properties
- Build LSTM response sequence analyzer for semantic anomaly detection
- Integrate anomaly scoring into Response Analysis Engine pipeline
- Tune false positive threshold on benchmark scan dataset

Days 19-20: LLM Integration

- Build LLM API client (Anthropic/OpenAI) with retry and rate limit handling
- Implement payload generation prompt templates per vulnerability class
- Implement finding explanation prompt for natural language report writing
- Build response parser for structured LLM output extraction

Day 21: Attack Graph Engine

- Design attack graph schema (nodes: assets/vulns, edges: exploitation relationships)
- Implement rule-based edge detection for common exploit chains (IDOR + XSS, CSRF + privilege escalation)
- Build graph visualization data exporter for frontend Cytoscape.js integration

Week 4: Platform & Polish (Days 22-30)

Days 22-23: Frontend Dashboard

- Build React app skeleton with Tailwind dark theme and Zustand state management
- Implement Target Management screen with scope visualization
- Build Live Scan Monitor with WebSocket connection to backend
- Implement Finding Detail screen with evidence viewer and PoC display

Days 24-25: Reporting System

- Build finding schema serializer with all required fields
- Implement PDF report generator (ReportLab or WeasyPrint)
- Implement JSON/CSV/Markdown exporters
- Build report template system with Executive and Technical report types

Days 26-27: Security Safeguards & Hardening

- Implement authorization confirmation flow with audit logging
- Add rate limiting middleware with burst protection
- Implement data encryption at rest for evidence storage
- Add sensitive parameter masking in all log outputs
- Implement scan impact minimization controls (read-only mode, payload blocklist)

Days 28-29: Integration Testing

- End-to-end scan test against intentionally vulnerable targets (DVWA, WebGoat, Juice Shop)
- Performance testing: verify scanner handles 50 concurrent targets without degradation
- False positive benchmark: measure FP rate against known-clean applications
- Security testing of the platform itself (eat own dog food)

Day 30: Documentation & Release Prep

- Write API documentation via FastAPI auto-docs + supplementary guides
- Write operator runbook: deployment, configuration, tuning guide
- Prepare Docker Compose and Kubernetes Helm chart for deployment
- Tag v1.0.0 release on repository with signed release artifacts

15. Future Extensions

The following capabilities are architected as extension points in the v1 platform and are planned for subsequent development cycles.

15.1 LLM-Based Autonomous Pentesting Agents

The next evolution of AWAP-AI is a fully autonomous agent layer where LLMs do not just generate payloads and explain findings — they actively plan and execute multi-step attack scenarios with full reasoning transparency. Each agent maintains a scratchpad of hypotheses about the target, selects attack actions based on observed evidence, and iterates toward exploit chain completion.

- Agent framework: LangGraph or custom FSM-based agent with tool-use capabilities
- Tools available to agent: recon queries, HTTP request sender, payload generator, response analyzer
- Chain-of-thought reasoning: agent outputs human-readable reasoning trace for each decision
- Human-in-the-loop mode: agent presents plan for approval before executing destructive probes

15.2 Graph-Based Attack Modeling

An advanced attack modeling system that maintains a persistent attack graph across multiple scan sessions. The graph evolves as new findings are discovered, enabling long-horizon exploit chain planning that spans multiple applications, services, and trust boundaries. Planned capabilities:

- Cross-service attack chains: SSRF in app A leads to credential access in service B
- Temporal attack modeling: vulnerabilities that require multiple steps with delays between them
- Attack simulation: replay confirmed exploit chains against patched targets to verify remediation

15.3 CI/CD Pipeline Integration

- GitHub Actions / GitLab CI native integration: security gate on every pull request
- Diff-based scanning: only test endpoints changed in the current PR diff
- Security regression detection: fail build if a previously remediated vulnerability re-appears
- SARIF output format: native integration with GitHub Security tab and VS Code Security extension
- Shift-left API: developers call AWAP-AI from their local dev environment for instant feedback

15.4 Cloud Security Scanning

- AWS security configuration assessment: IAM policies, S3 bucket policies, security group rules
- Azure / GCP configuration scanning via native SDKs
- Cloud metadata SSRF testing expanded to include full IMDS exploitation chains
- Serverless function endpoint discovery and vulnerability testing
- Container image scanning integration: Trivy API integration for known CVE detection

15.5 Advanced Evasion & Red Team Capabilities

- Full WAF bypass engine with ML-learned evasion per WAF vendor

- Browser fingerprint simulation: TLS fingerprint rotation, HTTP/2 fingerprint normalization
- Distributed scanning: route requests through rotating proxy pools to bypass IP-based rate limiting
- Traffic masquerading: blend scan traffic with legitimate user traffic patterns
- Custom scan profiles: researchers define their own attack module combinations and payload strategies

15.6 Collaboration & Platform Features

- Team workspaces: shared targets, findings, and reports with role-based access control
- Finding triage workflow: assign, escalate, accept-risk, verify workflows with notification
- Bug bounty submission automation: auto-format and submit findings to HackerOne/Bugcrowd APIs
- VDP (Vulnerability Disclosure Program) management: intake, triage, and tracking in one platform
- Knowledge base: auto-populated library of discovered vulnerabilities with remediation patterns

16. README & Developer Guide

Quick Start

```
# 1. Clone repository
git clone https://github.com/your-org/awap-ai.git
cd awap-ai

# 2. Copy environment configuration
cp .env.example .env
# Edit .env: set POSTGRES_PASSWORD, REDIS_PASSWORD, LLM_API_KEY, SECRET_KEY

# 3. Launch all services
docker-compose up -d

# 4. Run database migrations
docker-compose exec api alembic upgrade head

# 5. Create initial admin user
docker-compose exec api python -m awap.cli create-admin

# 6. Access dashboard
open http://localhost:3000
```

Repository Structure

```
awap-ai/
├── backend/
│   ├── awap/
│   │   ├── api/           # FastAPI routes, schemas, dependencies
│   │   ├── core/          # Config, database, security, celery
│   │   ├── engines/       # recon, crawler, attack, response, ai
│   │   ├── modules/       # Individual attack modules
│   │   ├── models/        # SQLAlchemy ORM models
│   │   ├── payloads/      # Payload databases and generators
│   │   └── reporting/     # Report generation
│   ├── tests/
│   ├── alembic/           # Database migrations
│   ├── Dockerfile
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── hooks/
│   │   ├── store/
│   │   └── api/
│   ├── Dockerfile
│   └── package.json
├── infra/
│   ├── docker-compose.yml
│   └── docker-compose.prod.yml
```

```
|   ├── kubernetes/      # Helm chart
|   └── terraform/       # Cloud deployment
├── ml/
|   ├── training/        # Model training scripts
|   ├── models/          # Serialized model files
|   ├── data/            # Training datasets
└── docs/                # Extended documentation
```

Environment Variables

Variable	Description
DATABASE_URL	PostgreSQL connection string
REDIS_URL	Redis connection string
SECRET_KEY	JWT signing key (generate: openssl rand -hex 32)
LLM_PROVIDER	anthropic openai
LLM_API_KEY	API key for configured LLM provider
MINIO_ENDPOINT	MinIO endpoint URL
MINIO_ACCESS_KEY	MinIO access key
SHODAN_API_KEY	Optional: Shodan API key for OSINT
CENSYS_API_ID	Optional: Censys API credentials
OAST_SERVER	OOB callback server endpoint
MAX_CONCURRENT_SCANS	Maximum parallel scans (default: 5)
DEFAULT_RATE_LIMIT	Requests/second per target (default: 10)

Adding a New Attack Module

```
# 1. Create module file
touch backend/awap/modules/my_vulnerability.py

# 2. Implement the AttackModule interface
from awap.engines.attack import AttackModule, Finding
from awap.core.types import VulnClass, Severity

class MyVulnerabilityModule(AttackModule):
    module_id = 'my_vulnerability'
    vuln_class = VulnClass.CUSTOM
    severity = Severity.HIGH
    requires = ['endpoints', 'params']

    def run(self, target, context) -> List[Finding]:
        findings = []
        for endpoint in context.endpoints:
            for payload in self.payload_engine.get(self.vuln_class):
                response = self.request(endpoint, payload)
                if self.analyze(response):
```

```
        findings.append(self.build_finding(endpoint, payload, response))
    return findings

    def verify(self, finding, context) -> bool:
        # Confirmation logic here
        return True

# 3. Register in module registry
# backend/awap/modules/__init__.py
from .my_vulnerability import MyVulnerabilityModule
MODULES.append(MyVulnerabilityModule)
```

Running Tests

```
# Unit tests
cd backend && pytest tests/unit/ -v

# Integration tests (requires running services)
pytest tests/integration/ -v

# End-to-end tests against DVWA
docker-compose -f docker-compose.test.yml up -d dvwa
pytest tests/e2e/ -v --target=http://localhost:4280

# Coverage report
pytest --cov=awap --cov-report=html tests/
```

Legal and Ethical Use

MANDATORY: This software is provided for authorized security testing only. You must have explicit written permission from the target system owner before using AWAP-AI. Unauthorized use is a criminal offense in most jurisdictions. The authors accept no liability for misuse.

- Always operate within your authorized scope
- Never test production systems without a signed rules-of-engagement document
- Respect program disclosure timelines when reporting to bug bounty programs
- Do not exfiltrate or retain sensitive data discovered during testing
- Report critical findings promptly to protect users of vulnerable systems

Appendix A: CVSS 3.1 Scoring Reference

Score Range	Severity	Priority	SLA	Response
9.0 - 10.0	CRITICAL	P0	4 hours	Immediate escalation
7.0 - 8.9	HIGH	P1	24 hours	Same-day review
4.0 - 6.9	MEDIUM	P2	72 hours	Sprint inclusion
0.1 - 3.9	LOW	P3	2 weeks	Backlog
0.0	INFO	P4	Next release	Documentation

Appendix B: Vulnerability Class to CWE Mapping

Vulnerability	CWE	OWASP 2021	CVSS Base (typical)
SQL Injection	CWE-89	A03:2021	9.8 (CRITICAL)
Cross-Site Scripting	CWE-79	A03:2021	6.1 (MEDIUM)
Command Injection	CWE-78	A03:2021	9.8 (CRITICAL)
SSRF	CWE-918	A10:2021	8.6 (HIGH)
XXE	CWE-611	A05:2021	7.5 (HIGH)
IDOR	CWE-284	A01:2021	6.5 (MEDIUM)
Broken Authentication	CWE-287	A07:2021	8.1 (HIGH)
SSTI	CWE-1336	A03:2021	9.8 (CRITICAL)
Path Traversal	CWE-22	A01:2021	7.5 (HIGH)
Deserialization	CWE-502	A08:2021	9.8 (CRITICAL)
HTTP Smuggling	CWE-444	A07:2021	8.1 (HIGH)
CORS Misconfiguration	CWE-942	A05:2021	6.5 (MEDIUM)

Appendix C: Scan Profiles Reference

Profile	Rate Limit	Depth	Modules	Use Case
Quick Scan	20 req/s	Shallow	Critical only	CI/CD gate
Standard	10 req/s	Medium	High+	Regular assessment
Full Scan	5 req/s	Deep	All modules	Full pentest

API Only	15 req/s	API-focused	API modules	API security review
Stealth	1 req/s	Deep	All modules	Evasion mode
Authenticated	10 req/s	Deep	All modules	Post-auth testing